

## TemaFinale26 – Sprint 0: Analisi dei Requisiti

### Requirements

Si riporta il **testo esatto del committente** (Protobook, cap. 31 – TemaFinale26):

*"A Maritime Cargo shipping company (from now on, simply company) intends to automate the operations of load of containers in the ship's cargo hold (or simply hold). To this end, the company plans to employ a Differential Drive Robot (from now, called cargorobot). The hold is a rectangular, flat area with an Input/Output port (IOPort). The area provides 4 slots to store the containers and a slot named slot5."*

- "• The slots1-4 depict the hold areas reserved to store one container each.*
- The slot5 depicts an area, where the cargorobot must temporarily store a container, before to place it in one of the slots1-4. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.*
- The IOPort is a device with a pushbutton and a display. The pushbutton is pressed by the customer in order to send a request to load a container on the cargo. The display is used to show the answer to the request and to show the current state of the hold.*
- The sensor associated to the IOPort is a device (a sonar) used to detect the presence of a container, when it measures a distance  $D$ , such that  $D < DFREE/2$ , during a reasonable time (e.g. 3 secs)."*

**TF2026 Requirements** – testo esatto:

*"The company asks us to build service named cargoservice that should work as follows. The cargoservice is able to receive a request to load a container sent by some customer by using the pushbutton of the IOPort.*

- It sends the answer retrylater, if the IOPort is currently occupied by a container or if the system is Out of service.*
- It rejects the request when the hold is already full, i.e. the slots1-4 are already occupied.*
- Otherwise, it considers the system as engaged, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led. When the request to load is accepted, the customer must move the container in the sensor area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes disengaged. Then, the cargoservice uses the cargorobot to move the container from the IOPort to the slot5 (for marking the container) and then to the reserved slot. The service must also show on the display on the IOPort:*
  - the current state of the hold;*
  - the message 'Service working', when it is all is going well;*
  - the message 'Out of service' if the sonar sensor measures a distance  $D > DFREE$  for at least 3 secs (perhaps a failure of the sonar)."*

## Layout dell'area (dalla figura allegata al tema)

```
+-----+
| HOME      [SONAR]          | HOME : posizione iniziale del cargorobot
| [SLOT1][SLOT2]            | SONAR : sensore in alto nell'area
| robot      [SLOT5]        | SLOT1..4 : aree di stoccaggio container
| [SLOT3][SLOT4]            | SLOT5 : area di marcatura temporanea
|
| [IOPORT]                  | IOPORT: pushbutton + display + sonar
+-----+
SENSOR ---- DFREE ---- | (parete/oggetto)
```

### Analisi dei requisiti

**Goal:** formalizzare le entita' del dominio escludendo le ambiguita' del linguaggio naturale (dichiarazioni Java per il dominio + modello qak per le interazioni).

### Entita' formalizzate

**Container** — merce da caricare; nel TF26 non ha peso ne' PID a monte: riceve un **barcode** solo durante la marcatura in slot5.

**Slot** — area di stoccaggio della stiva (slot1..slot4 di carico; slot5 di marcatura):

```
public class Slot {
    String name;          // "slot1".."slot4" | "slot5"
    int  posX, posY;      // posizione nella mappa
    boolean occupied;
    boolean isMarkingArea; // true solo per slot5
}
```

**Hold** — aggregato degli slot di carico (slot1..4) + slot5 di marcatura:

```
public class Hold {
    Slot[] slots;          // slot1..slot4 (+ slot5 marcatura)
    boolean ioportOccupied(); // container presente all'IOPort
    boolean isFull();       // tutti gli slot1..4 occupati
    String reserveFreeSlot(); // nome di uno slot libero, "" se pieno
    void  confirmStored(String slotName);
    void  freeSlot(String slotName);
}
```

**IOPort** — dispositivo composto: **pushbutton** (input richiesta), **display** (output: risposta + stato hold) e **sonar** (presenza container).

**Sonar (IOPort)** — misura la distanza D. Condizioni fissate dal committente:

```
container presente : D < DFREE/2 per ~3 s
sospetto guasto    : D > DFREE per ≥ 3 s
```

**Marker** — dispositivo in slot5 che etichetta il container con un barcode e **segnala** il completamento della marcatura.

**LED** — attuatore che **lampeggia** mentre il sistema e' *engaged*.

**CargoRobot** — il DDR fornito dal committente (risorsa data). Raggiunge posizioni obiettivo su comando; e' lo stesso robot gia' usato come **robotsmart**.

### Struttura: componenti logici

- il **cargoservice** (il software da costruire);
- il **cargorobot** (DDR fornito dal committente, gia' come **robotsmart**);
- i dispositivi dell'**IOPort**: pushbutton, display, sonar;
- il **marker** (in slot5) e il **LED**.

Trattandosi di componenti logicamente (micro)servizi a scambio di messaggi, formalizziamo la struttura con il linguaggio **qak**.

### Un modello dei requisiti

Il **cargoservice** e' il componente centrale: **proattivo** (gestisce la sessione di carico e comanda il **cargorobot**) e **reattivo** (al pushbutton, al sonar dell'IOPort, alla fine marcatura, al timeout). Lo modelliamo come QActor; **cargorobot**, sonar, marker e dispositivi IOPort sono entita' esterne con cui interagisce a scambio di messaggi.

```
System cargoservice26

//---- IOPort (pushbutton) -> cargoservice (richiesta con risposta)
Request loadrequest : loadrequest()          "pulsante premuto dal customer"
Reply  answer      : answer(ESP) for loadrequest "RESP = retrylater | reject | reserved(SLOT)"

//---- cargoservice -> IOPort (display)
Dispatch showOnDisplay : show(WHAT)          "risposta o stato hold sul display"

//---- osservazioni dal dominio (a cui il cargoservice reagisce)
Event  containerAtIOPort : containerAtIOPort() "sonar: D<DFREE/2 per ~3s"
Event  sonarFail         : sonarFail()         "sonar: D>DFREE per ≥3s"
Event  markingDone       : markingDone()       "il marker ha completato"

//---- cargoservice -> cargorobot (come in robotsmart26)
Request moverobot      : moverobot(TARGETX, TARGETY, STEPTIME)

//---- cargoservice -> LED
Dispatch ledBlink      : ledBlink(ONOFF)       "lampeggio mentre engaged"

QActor cargoservice context ctxcargo {
    //proattivo: pushbutton -> verifica stato/hold -> riserva slot -> comanda cargorobot
    //reattivo : containerAtIOPort, sonarFail, markingDone, timeout 30s
    ...
}
```

**Nota:** **come** il sonar produca gli eventi, **come** si realizzino pushbutton, display, marker e LED, e **come** il robot raggiunga le posizioni sono scelte della **fase di progetto**, non dei requisiti.

## Core Business e Goal del primo prototipo

**Domanda al committente:** e' corretto concordare che il 'core business' consiste anzitutto nel **portare un container, accettata la richiesta, dall'IOPort allo slot riservato passando per la marcatura in slot5**, mentre la gestione del display, del LED lampeggiante e del 'Out of service' sono aspetti da consolidare in iterazioni successive?

**Goal del primo prototipo** (fase Development). Realizzare il **cargoservice** che, ricevuta una richiesta valida (pushbutton) con stiva non piena e IOPort libero, riserva uno slot, attende il container all'IOPort (entro il tempo previsto), comanda il cargorobot a portarlo **IOPort → slot5 (marcatura) → slot riservato**, e riporta il robot a HOME, tornando pronto per una nuova richiesta.

Nel primo prototipo si possono semplificare gli output (display/LED come log a video) e rinviare la gestione completa di *Out of service* e del barcode a un'iterazione successiva.

## Domande aperte al committente

- D1.** Posizioni: quali sono le coordinate (nella mappa) di IOPort, HOME, slot1..slot5? (la figura mostra il layout, ma servono le posizioni esatte; cfr. tf25map e Robotsmart26Cmds, dove IOPort=(4,0))
- D2.** "IOPort occupied by a container": come si rileva che l'IOPort e' occupato — e' lo stesso sonar ( $D < D_{FREE}/2$ ) o un'informazione distinta?
- D3.** Timeout 30s: parte alla risposta di slot riservato e se scade il sistema torna 'available' liberando lo slot riservato? Il container eventualmente arrivato resta?
- D4.** Marcatura: il 'marker' e' un dispositivo reale/simulato? Il barcode prodotto va memorizzato o basta il segnale 'markingDone'?
- D5.** Display: e' un dispositivo fisico, una pagina web, o (per la demo) un output a video? Quali messaggi esatti mostrare per "current state of the hold"?
- D6.** Stato 'Out of service' (sonarFail): le richieste in arrivo vanno tutte rifiutate con retrylater, e il servizio riprende automaticamente quando  $D \leq D_{FREE}$ ?
- D7.** Pushbutton: una sola richiesta per volta (mentre engaged le altre danno retrylater)?

## Problematiche aperte (per l'analista)

### **P1 — Realizzazione dei dispositivi senza hardware fisico.**

Non disponendo del PicoW fisico, sonar, pushbutton, display, marker e LED vanno realizzati in modo **simulato** (in Python/Java): il sonar tramite il mock `mockPicowRadar.py` (stesso messaggio MQTT del PicoW reale), gli altri come input/output a video o eventi simulati. I contratti (i messaggi) restano identici a quelli di un'eventuale realizzazione hardware.

### **P2 — Sorgente del sonar.**

Il sonar dell'IOPort puo' provenire dal mock (MQTT) oppure dal sonar simulato in WEnv (evento `sonaralarm` di `robotmnemo`, come in `robotsmart26tf26.qak`): le due sorgenti sono intercambiabili perche' il `cargoservice` reagisce allo stesso evento logico.

### **P3 — Movimento IOPort → slot5 → slot.**

Il deposito e' modellato in modo astratto: il container si considera spostato quando il `cargorobot` **raggiunge** la posizione (slot5 e poi lo slot), tramite `moverobot(X,Y)`, senza richiedere un attuatore di presa (non presente nel DDR).

### **P4 — Granularita' del Test Plan automatizzato (richiesto dalla valutazione).**

Caso minimo significativo: una richiesta accettata che attraversa l'intero ciclo (`pushbutton` → `slot riservato` → `container all'IOPort` → `IOPort` → `slot5` → `slot` → `HOME`), verificando alla fine che lo stato della stiva sia coerente. Casi aggiuntivi: `hold pieno (reject)`, `IOPort occupato/out of service (retrylater)`, `timeout (disengaged)`.



By Alexander Kreuzer

Email: [Alexander.kreuzer@studio.unibo.it](mailto:Alexander.kreuzer@studio.unibo.it)

GIT repo: [https://github.com/Alexing-uni/ISS26\\_Alexander\\_Kreuzer](https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer)